DISEÑO DE SISTEMAS DE INFORMACIÓN

“METAMAPA”

JUSTIFICACIONES DE DISEÑO

Trabajo Práctico Anual Integrador
-2025-

Grupo N°5



ENTREGA 4: Diseño y Maquetado de Interfaz de Usuario

Landing Page (Todos)

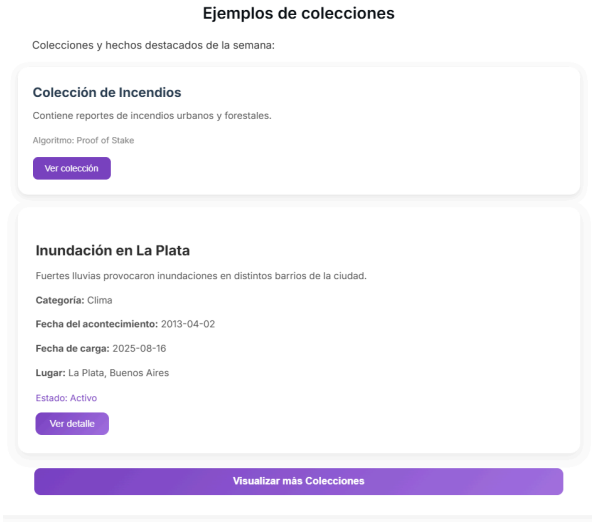
1. El sistema debe contar con una *landing page* accesible públicamente que permita:
    - Presentar el propósito y objetivos de MetaMapa.

Se realizó una interfaz **clara y ordenada**, con buena **jerarquía visual**, **lenguaje simple** y diseño **minimalista** que facilita la lectura y comprensión rápida del propósito y objetivos



- Mostrar ejemplos destacados de colecciones y hechos.

Es **intuitivo y accesible**, permitiendo al usuario identificar y explorar colecciones sin fricción. Se invita a conocer más hechos y explorar la página. Esta sección puede incluso reunir los hechos más destacados de la semana, atrayendo el interés de los visualizadores.



Además, incluir enlaces de acceso a:

- Visualización de hechos sin login.

Desarrollamos un *botón "**Colecciones**" con ese fin,* a donde se puede acceder sin estar logueado. El usuario es llevado a una nueva pestaña (colecciones.html).

- Registro o inicio de sesión para contribuyentes y administradores.

Lo cumplimos mediante un *botón **"Login"***. Abre un modal, donde se valida la información personal del usuario y se le permite iniciar sesión con su cuenta.

- Información legal y de privacidad.

Lo cumplimos mediante un footer con link a "***Información legal y de privacidad***" . Al ingresar, muestra toda la información en una nueva pestaña (acerca.html).

Aquí se muestra en forma general la página de landing:



Todo usuario que ingrese es dirigido al index.html. Se trata de la página principal. Es posible leer acerca de MetaMapa, su propósito y objetivos.

Todas las pestañas garantizan acceso fácil a la barra de navegación con posición fija para el desplazamiento. Se tiene acceso al botón de "*ver colecciones*" y "*login*" en todo momento, invitando al usuario a recorrer el sitio. El sistema de rutas y jerarquías permite acceder a funcionalidades clave con una profundidad de navegación máxima de tres niveles.

Para mantener la consistencia, todas las pestañas poseen el mismo footer, con link a la pestaña acerca.html, la cual posee toda la información legal correspondiente. El usuario puede consultarla en todo momento.

El footer de información legal se encuentra así:



Y al acceder a “acerca.html” se puede ver la siguiente información:



En cada pestaña y botón se asegura coherencia visual en *tokens* de diseño. Esto se hizo respetando la paleta de colores elegida y las tipografías seleccionadas.

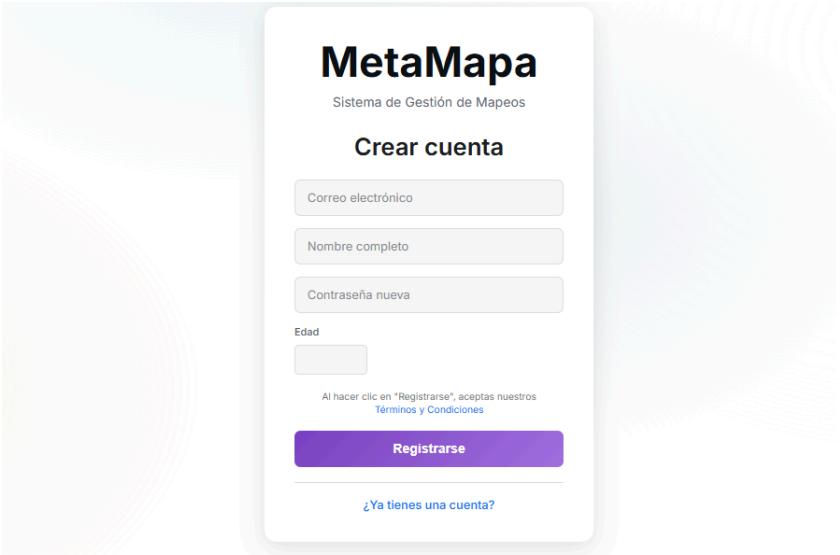
Registro de Usuario (Todos)

Requerimientos evaluados:
4. El sistema debe permitir el registro de contribuyentes con nombre obligatorio, y otros datos opcionales.

Si el usuario no tiene cuenta, puede crearse una. El sistema lo dirige a la pestaña registro.html. Para crear usuario, se realiza una request HTTP POST a UsuarioController del módulo Fuente Dinámica. Al crearse una cuenta, todo usuario es un Contribuyente no anónimo.

Datos obligatorios requeridos:

- Correo electrónico (username usado para login, es único).
- Nombre completo
- Contraseña (permite validar el login).
- Edad (para saber si luego puede subir hechos).



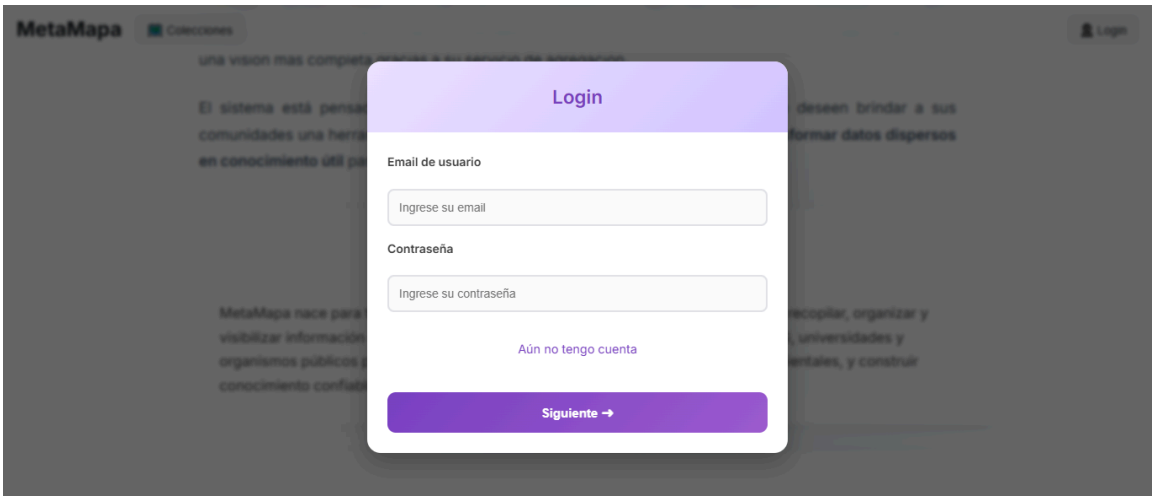
Inicio de sesión (Todos)

5. El sistema debe permitir el inicio de sesión de contribuyentes y administradores.

Una vez creada la cuenta, puede ingresar a ella e iniciar sesión. El sistema abre un modal de “login”. Para validar al usuario, se realiza una request HTTP POST a UsuarioController del módulo Fuente Dinámica.

Datos requeridos:

- Correo electrónico (username usado para login, es único).
- Contraseña (permite validar el login).



Visualización de Colecciones y Hechos (todos)

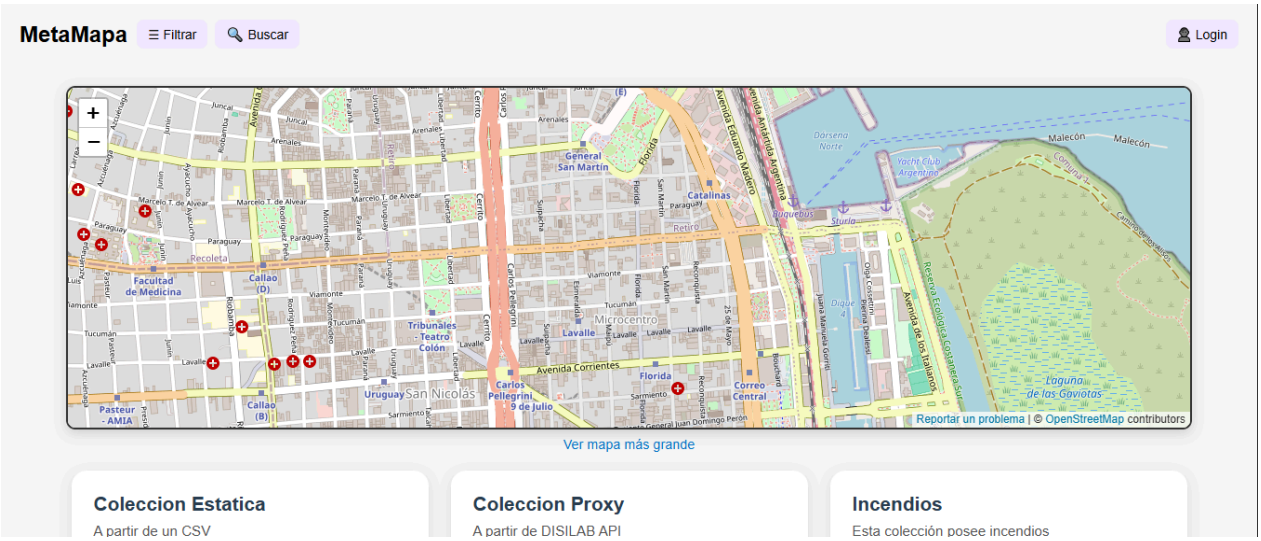
Requerimientos evaluados:

2. El sistema debe permitir el acceso anónimo para visualizar hechos y colecciones.

3. El sistema debe mostrar los hechos en un mapa interactivo y permitir ver sus detalles al hacer clic en cada marcador.

6. El usuario visualizador debe poder acceder a un listado de colecciones disponibles.

En la pestaña colecciones.html, el usuario podrá ver un **mapa dinámico**, donde serán cargados los hechos de los usuarios en tiempo real. Esto le permite desplazarse por él, con la posibilidad de hacer click sobre los hechos que aparecen en pantalla.



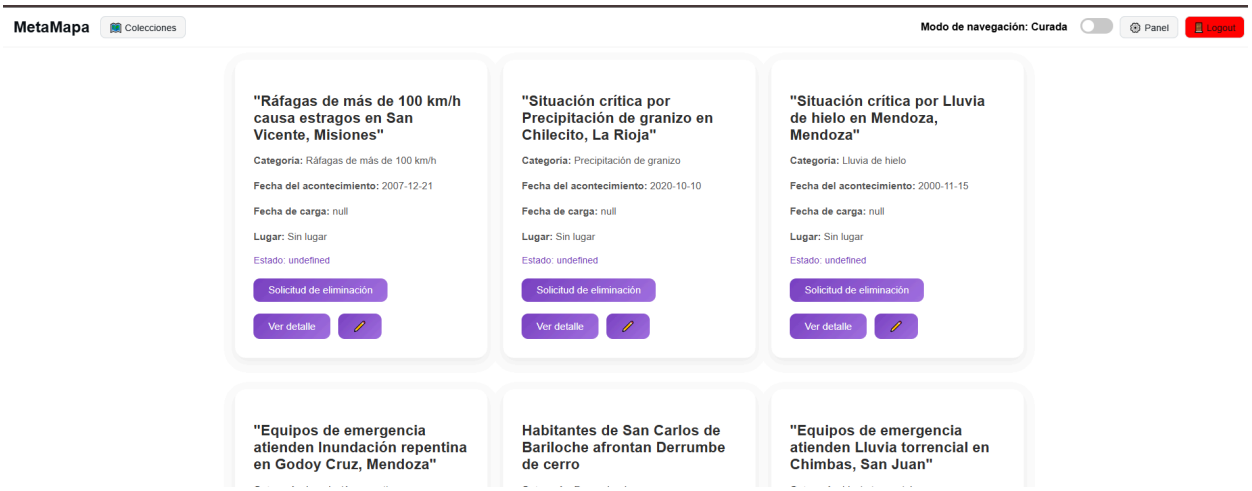
Debajo del mapa, podrá conocer las colecciones cargadas por los administradores del sistema. Estas colecciones son accesibles para cualquier usuario, logueado o no. Las colecciones muestran Título, Descripción y el algoritmo utilizado.



Para desplegar las colecciones dinámicamente de forma prolija, se utilizan grillas modeladas con CSS. En computadoras es posible ver hasta tres colecciones por fila, mientras que en dispositivos móviles se ve de a una por fila. Esto es comúnmente utilizado en la mayoría de aplicativos conocidos por el usuario.

Visualizar Hechos (Todos)

El botón “Ver colección”, nos lleva a la pestaña hechos.html, encargada de mostrar los hechos obtenidos a partir de las fuentes de la colección seleccionada.



Los hechos se obtienen mediante HTTP GET a ColeccionController, en el módulo Servicio Agregador. El ColeccionService se encarga de obtener todos los hechos de cada una de las fuentes de la colección en particular.

Para desplegar los hechos dinámicamente de forma prolija, se utilizan grillas modeladas con CSS. En computadoras es posible ver hasta tres hechos por fila, mientras que en dispositivos móviles se ve de a uno por fila.

9. El usuario visualizador debe poder visualizar detalles de un hecho, incluyendo ubicación, contenido multimedia y fuente.

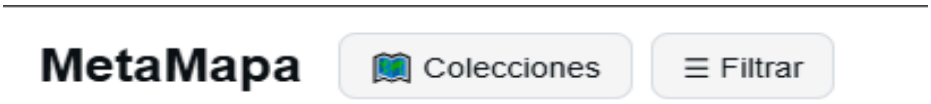
Es posible ver los detalles de los hechos, mostrando ubicación, el contenido multimedia y la fuente al hacer click sobre el botón “***Ver Detalle***”.



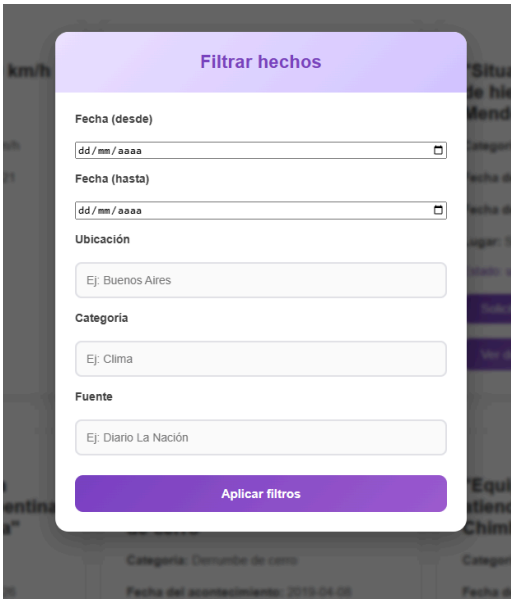
Filtrar Hechos (Contribuyente y Visualizador)

Requerimientos evaluados:

7. El usuario visualizador debe poder filtrar los hechos dentro de una colección por fecha, ubicación, categoría y fuente.



Se incluyó en la barra de navegación un botón "***Filtrar***", que permite aplicar filtros en los hechos visualizados. Al clickear muestra un modal que permite agregar todos los filtros que se mencionan en el requerimiento.

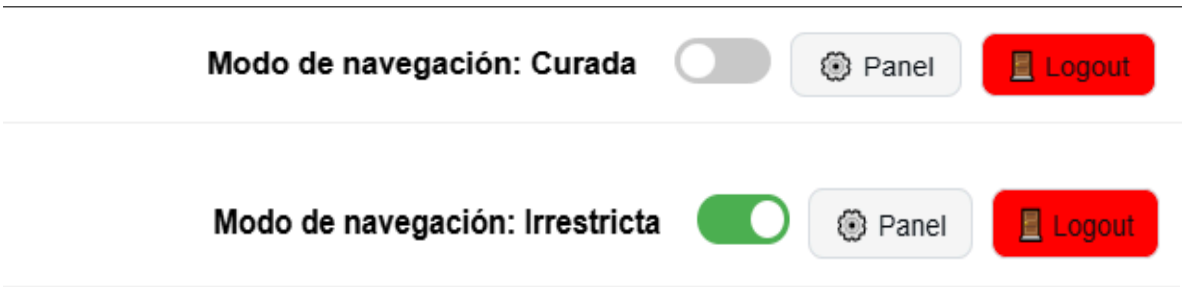


Cambiar modo de navegación (Contribuyente y Visualizador)

Requerimientos evaluados:

8. El usuario visualizador debe poder alternar entre modo de navegación curado e irrestricto.

Esto permite cambiar el modo de navegación siempre que el usuario quiera al ver hechos dentro de una colección.



Crear Solicitud de Eliminación (Contribuyente y Visualizador)

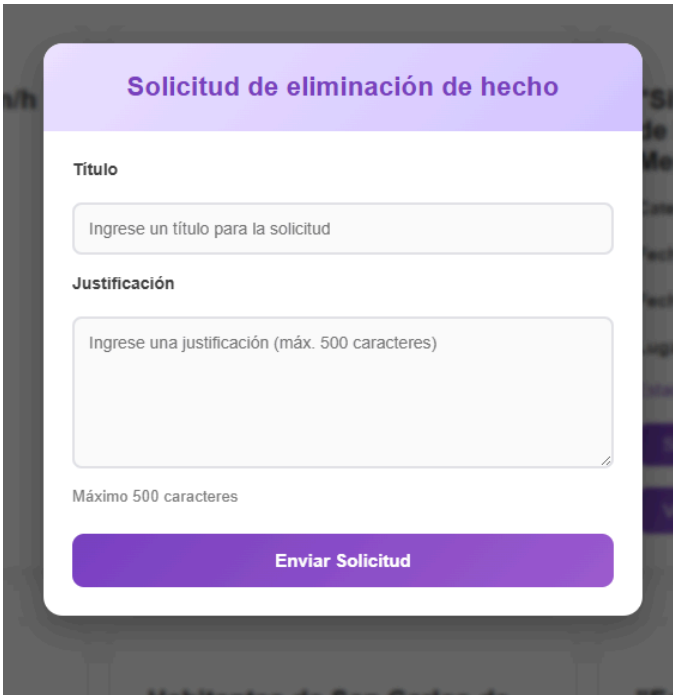
Requerimientos evaluados:
10. El usuario visualizador debe poder generar solicitudes de eliminación de hechos con una justificación mínima de 500 caracteres.

13. El contribuyente debe poder generar solicitudes de eliminación de hechos existentes.

El contribuyente debe poder generar solicitudes de eliminación de hechos existentes.

Esto lo resolvemos mediante un botón dentro del hecho que permite generar una solicitud de eliminación para ese hecho particular. Este despliega un modal en donde se pedirá título y una justificación de por qué desea eliminar el hecho. Además, se le aclara que debe ser de máximo 500 caracteres.



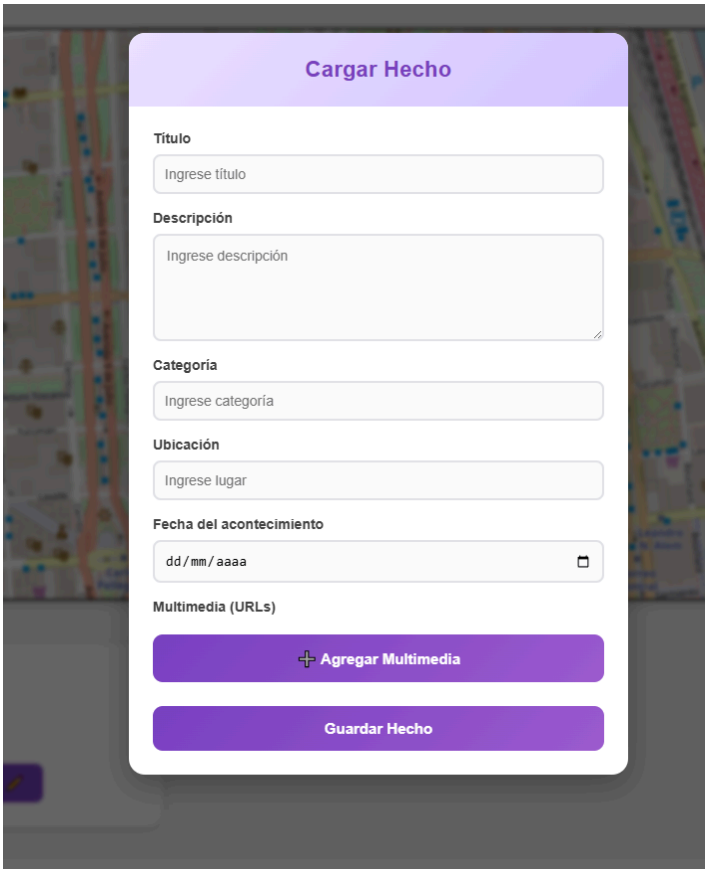


Carga de Hechos (Contribuyente)

Requerimientos evaluados:

11. El contribuyente (registrado o anónimo) debe poder subir nuevos hechos a través de un formulario que permita adjuntar título, descripción, categoría, ubicación, fecha, y multimedia.

Permitimos que el contribuyente pueda cargar hechos desde la página principal, de colecciones. Luego se agregarán a su respectiva colección, mediante la fuente dinámica, cuando los revise el administrador.



Editar hechos (Contribuyente)

Requerimientos evaluados:
12. El contribuyente registrado debe poder editar los hechos que haya creado, dentro de un plazo de 7 días.

Resolvemos este requerimiento con el clásico botón de editar con el icono de un lápiz, para que el usuario pueda identificarlo fácilmente. A nivel de usabilidad es intuitivo. El back debe realizar las validaciones correspondientes, para informar al front si se encuentra dentro del plazo correspondiente. En caso de estar fuera del plazo, el usuario jamás podrá ver el botón de edición.



Panel de Control (Administrador)

Requerimientos evaluados:
14. El administrador debe poder acceder a un panel de control con resumen de actividad (hechos, fuentes, solicitudes de eliminación).



Modificar Colecciones (Administradores)

Requerimientos evaluados:
15. El administrador debe poder crear, editar y eliminar colecciones

Quienes estén logueados y posean el rol ADMINISTRADOR, podrán además visualizar botones de edición. En esta iteración es posible:
Crear: HTTP POST a ColeccionController, en el módulo Servicio Agregador.
Editar: HTTP PUT a ColeccionController.
Eliminar: HTTP DELETE a ColeccionController.
Recordemos que el Servicio Agregador es el encargado de administrar y persistir las colecciones.



Carga de Colecciones (Administradores)

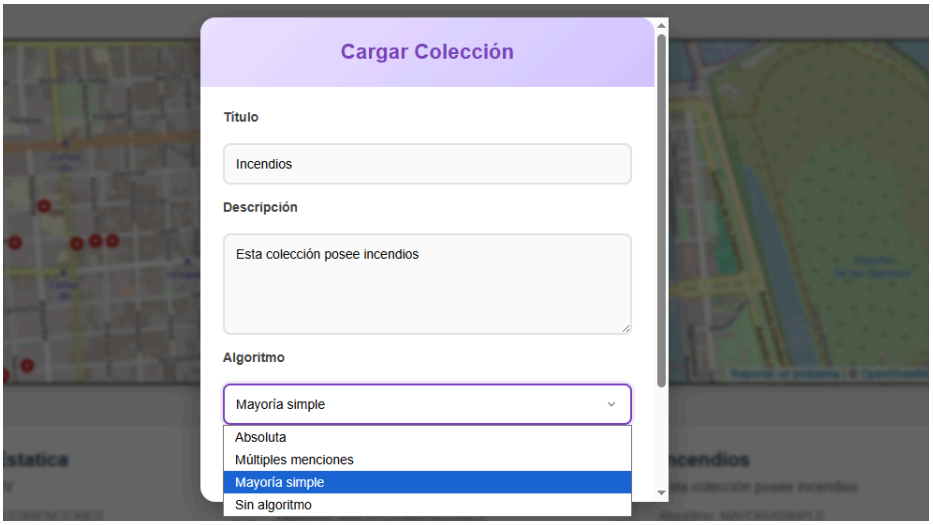
Requerimientos evaluados:
16. El administrador debe poder configurar fuentes (estáticas, dinámica y proxy) asociadas a una colección.
17. El administrador debe poder configurar el algoritmo de consenso para cada colección.

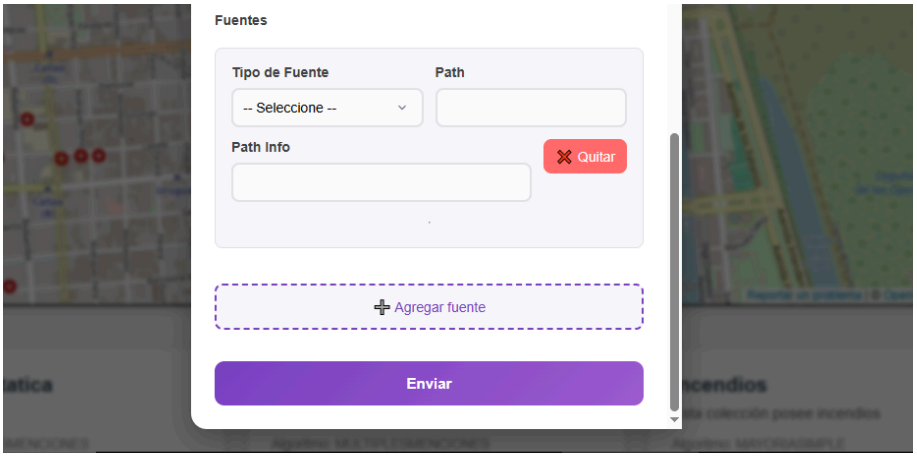
La carga de colecciones, abre un modal con los siguientes requisitos:

- Título.
- Descripción.
- Algoritmo de Consenso (Enum).
- Fuentes. Pudiendo agregar más de una.

Para cada fuente se pide:

- Tipo de fuente (Enum. DINÁMICA, PROXY o ESTÁTICA).
- Path. Es la dirección de la API. Ejemplo: localhost:8080/api
- PathInfo: Es el nombre o dirección del recurso a utilizar: Para la estática, debe ser un dataset, por ejemplo CSV, desastres_naturales.csv. Para la proxy, debe ser la API que proveerá los hechos. Ejemplo: disilab. Para la fuente dinámica no se requiere este campo, y puede ser nulo.





Manejo de Solicitudes (Administradores)

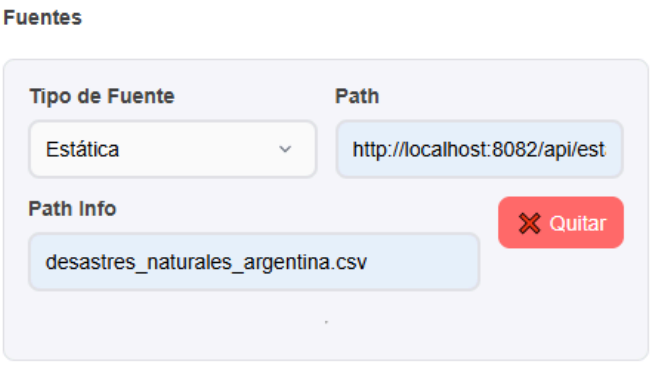
18. El administrador debe poder aprobar, rechazar o aceptar modificaciones de los hechos subidas por contribuyentes.

19. El administrador debe poder aprobar o rechazar solicitudes de eliminación de hechos



20. El administrador debe poder importar hechos desde archivos CSV con más de 10.000 entradas.

Esto es posible mediante el uso del módulo Fuente Estática siempre y cuando la ruta del archivo CSV sea encontrada por el backend.



Se hicieron las pruebas correspondientes y se pudieron obtener más de 3.000 hechos para esta fuente particular.

Normalizador

Los títulos de los hechos deberán tener un formato normalizado para mantener una consistencia en la base de datos. Para lograr esto dentro del módulo de servicio agregador hay una clase final que se encarga de múltiples funciones para asegurar la correcta normalización.

```
public boolean estaDuplicado(final String texto) throws IOException, InterruptedException {
    final String textoAVerificar = mapear(texto);
    if(conjuntoTitulos.isEmpty()) {
        conjuntoTitulos.add(textoAVerificar);
        return false; //se agrego y listo
    }
    //tenemos que convertir a json nuestro input para mandar a la API

    Map<String, Object> inputs = new HashMap<>();
    inputs.put("source_sentence", textoAVerificar);
    inputs.put("sentences", conjuntoTitulos);

    Map<String, Object> payload = new HashMap<>();
    payload.put("inputs", inputs);

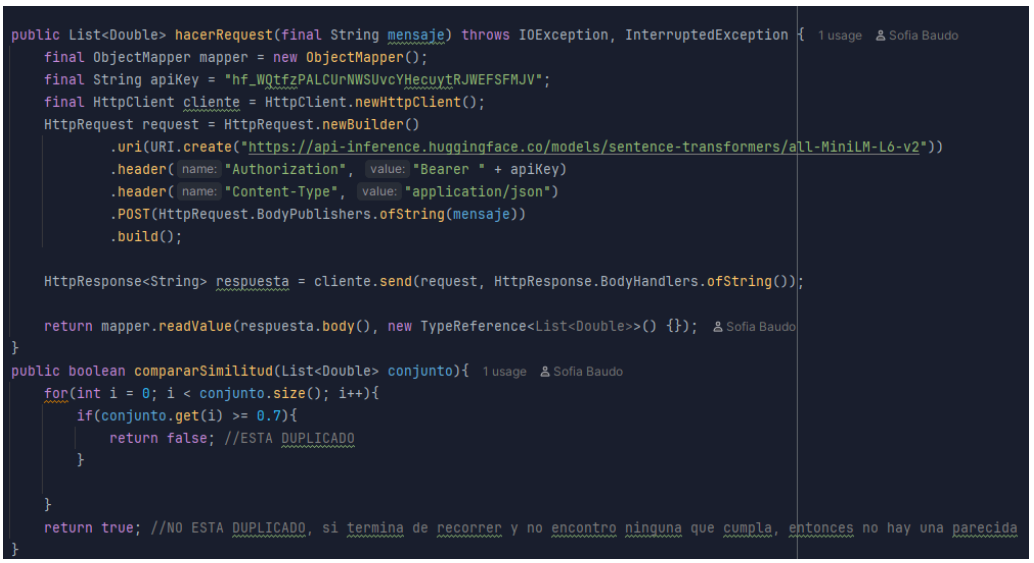
    ObjectMapper mapper = new ObjectMapper();
    //convertimos el objeto payload a Json para mandarle
    String requestBody = mapper.writeValueAsString(payload);
    List<Double> conjuntoFinal = hacerRequest(requestBody);
    if(compararSimilitud(conjuntoFinal)){
        conjuntoTitulos.add(textoAVerificar);
        return false;
    }
    else{
        return true;
    }
}
```

Para la implementación del normalizador se tiene que realizar consultas a una API de una LLM. La selección entre diferentes LLM es trivial, sin embargo, la aplicación de la misma es esencial para determinar similitudes y diferencias sintácticas, pero también semánticas.

Usamos esta API: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 que se encarga, como dijimos anteriormente, de analizar la relación semántica entre diferentes textos dados. Además, antes de llamar a la API, “mapeamos” para que este toda la frase sin espacios innecesarios y sin mayúsculas.

El normalizador tiene como objetivo devolver False si es que no está duplicado y True si es que está duplicado, teniendo en cuenta el conjunto de títulos de los hechos. Cuando se considera que el hecho no está duplicado, se agrega el título del mismo al conjunto de títulos para poder compararlo a futuro con otros hechos nuevos.

Nuestra intención es que a la hora de persistir un hecho en alguna fuente, verificar con el normalizador si es un hecho duplicado para no persistir y ocupar espacio en memoria innecesariamente.



```java
public List<Double> hacerRequest(final String mensaje) throws IOException, InterruptedException {  1 usage  Sofia Baudo
    final ObjectMapper mapper = new ObjectMapper();
    final String apiKey = "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";
    final HttpClient cliente = HttpClient.newHttpClient();
    HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api-inference.huggingface.co/models/sentence-transformers/all-MiniLM-L6-v2"))
            .header( name: "Authorization",  value: "Bearer " + apiKey)
            .header( name: "Content-Type",  value: "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(mensaje))
            .build();

    HttpResponse<String> respuesta = cliente.send(request, HttpResponse.BodyHandlers.ofString());

    return mapper.readValue(respuesta.body(), new TypeReference<List<Double>>() {});  Sofia Baudo
}
public boolean compararSimilitud(List<Double> conjunto){  1 usage  Sofia Baudo
    for(int i = 0; i < conjunto.size(); i++){
        if(conjunto.get(i) >= 0.7){
            return false; //ESTA DUPLICADO
        }

    }
    return true; //NO ESTA DUPLICADO, si termina de recorrer y no encontro ninguna que cumpla, entonces no hay una parecida
}
```

En esta imagen se puede ver el método en donde se realiza la request con la API y el método donde se compara si el número obtenido por la API supera o no el umbral de similitud semántica, 0.7.

MetaMapa - Endpoints API REST

Fuente Dinámica

HECHOS

Crear Hecho
POST http://localhost:8080/api/dinamica

JSON input:

```
{
  "titulo": "Inundación en Buenos Aires",
  "descripcion": "Fuertes lluvias en la ciudad",
  "categoria": "Inundación",
  "fechaAcontecimiento": "2023-11-12",
  "lugarAcontecimiento": {
    "latitud": -34.6037,
    "longitud": -58.3816
  },
  "etiquetas": [
    "lluvia",
    "emergencia",
    "clima"
  ]
}
```

Crear Hecho desde Usuario
POST http://localhost:8080/api/dinamica/{idUsuario}

El usuario debe existir y estar registrado.

Obtener Hechos
GET http://localhost:8080/api/dinamica

Obtener Hecho por ID
GET http://localhost:8080/api/dinamica/{idHecho}

JSON output:


```
{
        "titulo": "Inundación en Buenos Aires",
        "descripcion": "Fuertes lluvias en la ciudad",
        "categoria": "Inundación",
        "fechaAcontecimiento": "2023-11-12",
        "lugarAcontecimiento": {
            "latitud": -34.6037,
            "longitud": -58.3816
        },
        "etiquetas": [
            "lluvia",
            "emergencia",
            "clima"
        ],
        "creador": null
    }
```


Editar Hecho por ID

PUT http://localhost:8080/api/dinamica/{idHecho}

JSON input:


```
{
  "titulo": "Inundación actualizada en Buenos Aires",
  "descripcion": "Actualización: trabajan en el lugar.",
  "categoria": "Inundación",
  "fechaAcontecimiento": "2023-11-12",
  "etiquetas": ["lluvia", "emergencia", "actualizado"]
}
```


Si **no** es editable, retornará *"status": 403, "error": "Forbidden". No autorizado*

Eliminar Hecho por ID

DELETE http://localhost:8080/api/dinamica/{idHecho}

USUARIOS

Crear Usuario
POST http://localhost:8080/api/dinamica/usuarios

Por defecto, todo Usuario registrado es CONTRIBUYENTE. Le permite comenzar a cargar hechos de forma no anónima.

JSON input:
```
{
  "nombre": "Juan Perez",
  "password": “clave123",
  "email": "juan.perez@email.com",
  "edad": 30
}
```

Obtener Usuarios
GET http://localhost:8080/api/dinamica/usuarios

Obtener Usuario por ID
GET http://localhost:8080/api/dinamica/usuarios/{idUsuario}

JSON output:
```
{
    "id": 1,
    "nombre": "Juan Perez",
    "rol": "CONTRIBUYENTE"
}
```

Modificar Usuario por ID
PUT http://localhost:8080/api/dinamica/usuarios/{idUsuario}

JSON input:
```
{
  "password": "segura567"
}
```

Permite modificar opcionalmente los campos que se deseen.

Login de Usuario
POST http://localhost:8080/api/dinamica/usuarios/login

JSON input:
```
{
    "usuario":"juan.perez@email.com",
    "password":"segura123"
}
```

JSON output:


```
{
   "id": 1,
   "nombre": "Juan Perez",
   "rol": "CONTRIBUYENTE"
}
```


Si **no** es correcto, retornará *"status": 401, "error": "Unauthorized".*
Caso correcto, el login retorna 200 OK.

Fuente Proxy

HECHOS - API

Obtener Hechos de API
GET http://localhost:8083/api/proxy/{nombreAPI}

Se desarrolló un modelo *Strategy* que actúa de acuerdo a la API seleccionada. Debe estar soportada en el sistema.

JSON output:
Se retorna un arreglo de hechos:


```
 {
       "titulo": "Situación crítica por Precipitación de granizo en Chilecito, La Rioja",
       "descripcion": "Especialistas evalúan los daños...",
       "categoria": "Precipitación de granizo",
       "fechaAcontecimiento": "2020-10-10",
       "lugarAcontecimiento": {
          "latitud": -29.19831,
          "longitud": -67.533537
       },
       "etiquetas": null
   },
```


Obtener Hechos de API por ID
GET http://localhost:8083/api/proxy/{nombreAPI}/{idHecho}

Si la API en cuestión lo soporta, permite traer un hecho por id.

Fuente Estática

HECHOS - DATASET

Obtener Hechos de DATASET
GET http://localhost:8082/api/estatica/{tipo}/{archivo}

Se debe especificar el tipo de DATASET (Bases de Datos, CSV, XLSX, etc.). Con patrón *Factory* se crea una subclase concreta (DatasetCsv, DatasetDb, etc.) y en consecuencia, sus métodos de obtención de hechos. Se indica además la ruta de información para el dataset puntual.

Servicio Agregador

COLECCIÓN

Crear Colecciones
POST http://localhost:8081/api/colecciones

JSON input:

```
{
  "titulo": "C.ESTATICA",
  "descripcion": "Colección estatica",
  "algoritmoConsenso": "MAYORIASIMPLE",
  "fuentes": [
    {
      "path": "http://localhost:8082/api/estatica/CSV",
      "pathInfo": "desastres_naturales_argentina.csv",
      "tipoFuente": "ESTATICA"
    }
  ]
}
```

Obtener colecciones
GET http://localhost:8081/api/colecciones

Obtener colección por ID
GET http://localhost:8081/api/colecciones/{idCol}

JSON output:

```
{
    "id": 1,
    "titulo": "C.ESTATICA",
    "descripcion": "Colección estatica",
    "algoritmoConsenso": "MAYORIASIMPLE"
}
```

Editar colección por ID
PUT http://localhost:8081/api/colecciones/{idCol}

JSON input:
```
{
  "titulo": "Nombre editado",
  "descripcion": "Pepe y Batman son la misma persona"
}
```

Borrar colección por ID
DELETE http://localhost:8081/api/colecciones/{idCol}

Obtener Hechos de una colección
GET http://localhost:8081/api/colecciones/{idColeccion}/hechos?curada=boolean

Obtiene los hechos de cada una de las fuentes que forman parte de la colección. Sí curada es true, se traen solo aquellos hechos que cumplan con el algoritmo de consenso establecido para la colección.

Modificar algoritmo de consenso
PUT http://localhost:8081/api/colecciones/{idCol}/algoritmo-consenso?algoritmo=nomAlgorit

Se debe ingresar un algoritmo soportado:
```
public enum TiposAlgoritmos {
   ABSOLUTA,
   MULTIPLESMENCIONES,
   MAYORIASIMPLE,
   NOHAYALGORITMO
}
```

Agregar fuente a colección
POST http://localhost:8081/api/colecciones/{idCol}/fuentes/{idFuente}

Quitar fuente
DELETE http://localhost:8081/api/colecciones/{idCol}/fuentes/{idFuente}

Ambas modifican la tabla intermedia "colección por fuente".

FUENTE

Crear fuente
POST http://localhost:8081/api/fuentes

JSON input:
```
{
  "tipoFuente": "ESTATICA",
  "path": "http://localhost:8082/api/estatica/CSV",
  "pathInfo": "desastres_tecnologicos_argentina.csv"
```

}

Obtener fuentes
GET http://localhost:8081/api/fuentes

Obtener fuente por ID
GET http://localhost:8081/api/fuentes/{id}

JSON output:

{
  "tipoFuente": "ESTATICA",
  "path": "http://localhost:8082/api/estatica/CSV",
  "pathInfo": "desastres_naturales_argentina.csv"
}

Editar fuente
PUT http://localhost:8081/api/fuentes/{id}

Eliminar fuente
DELETE http://localhost:8081/api/fuentes/{id}

SOLICITUD

Crear solicitud
POST http://localhost:8081/api/solicitudes/eliminacion

JSON input:

{
 "idHecho": 123,
 "motivoBorrado": "Información incorrecta"
}

JSON output:

{
  "id": 1,
  "idHecho":123,
  "motivoBorrado": "Información incorrecta",
  "estado": "PENDIENTE"
}

Por defecto, se crea la solicitud y se marca como PENDIENTE. Siempre que se crea una solicitud se revisa si cumple con las condiciones. En caso de que el detector de spam considere el mensaje como basura, se marca por defecto como RECHAZADA.

Crear solicitud
GET http://localhost:8081/api/solicitudes/eliminacion

Obtener solicitudes
GET http://localhost:8081/api/solicitudes/eliminacion

PUT http://localhost:8081/api/solicitudes/eliminacion/7/denegar

Servicio de Estadísticas - Documentación de API

URL base: /estadísticas

1- Obtener estadísticas por tipo:

Devuelve todas las estadísticas almacenadas de un tipo específico
Endpoint: GET /estadisticas/{tipo}

Tipo de estadística a consultar. Debe coincidir con alguno de los valores del enum TipoEstadistica: PROVINCIA_TOP_COLECCION, CATEGORIA_TOP_COLECCION, PROVINCIA_TOP_CATEGORIA_COLECCION, HORA_TOP_CATEGORIA_COLECCION, SOLICITUDES_SPAM_COLECCION

Respuesta exitosa (200 OK):

```
[
  {
    "id": 1,
    "tipo": "CATEGORIA_TOP_COLECCION",
    "clave": "Deportes",
    "valor": 45,
    "fechaCalculo": "2025-09-07T00:00:00"
  },
  ...
]
```

2- Exportar estadísticas a CSV:

Exporta las estadísticas de un tipo específico en formato CSV
Endpoint: GET /estadisticas/export/{tipo}

Respuesta exitosa (200 OK):

- Tipo de contenido: **text/csv**
- Contenido ejemplo:
  clave,valor,fecha
  Deportes,45,2025-09-07T00:00:00
  Cultura,32,2025-09-07T00:00:00

3- Recalcular estadísticas de una colección:

Recalcula todas las estadísticas asociadas a una colección específica. Se le pide en el body una lista de categorías de la colección para las que se recalculan estadísticas por categoría. Puede ser vacía si se quiere recalcular todo

Cuerpo de ejemplo (JSON):

```
[
  "Deportes",
  "Cultura",
  "Tecnología"
]
```

Respuesta exitosa: 200 OK

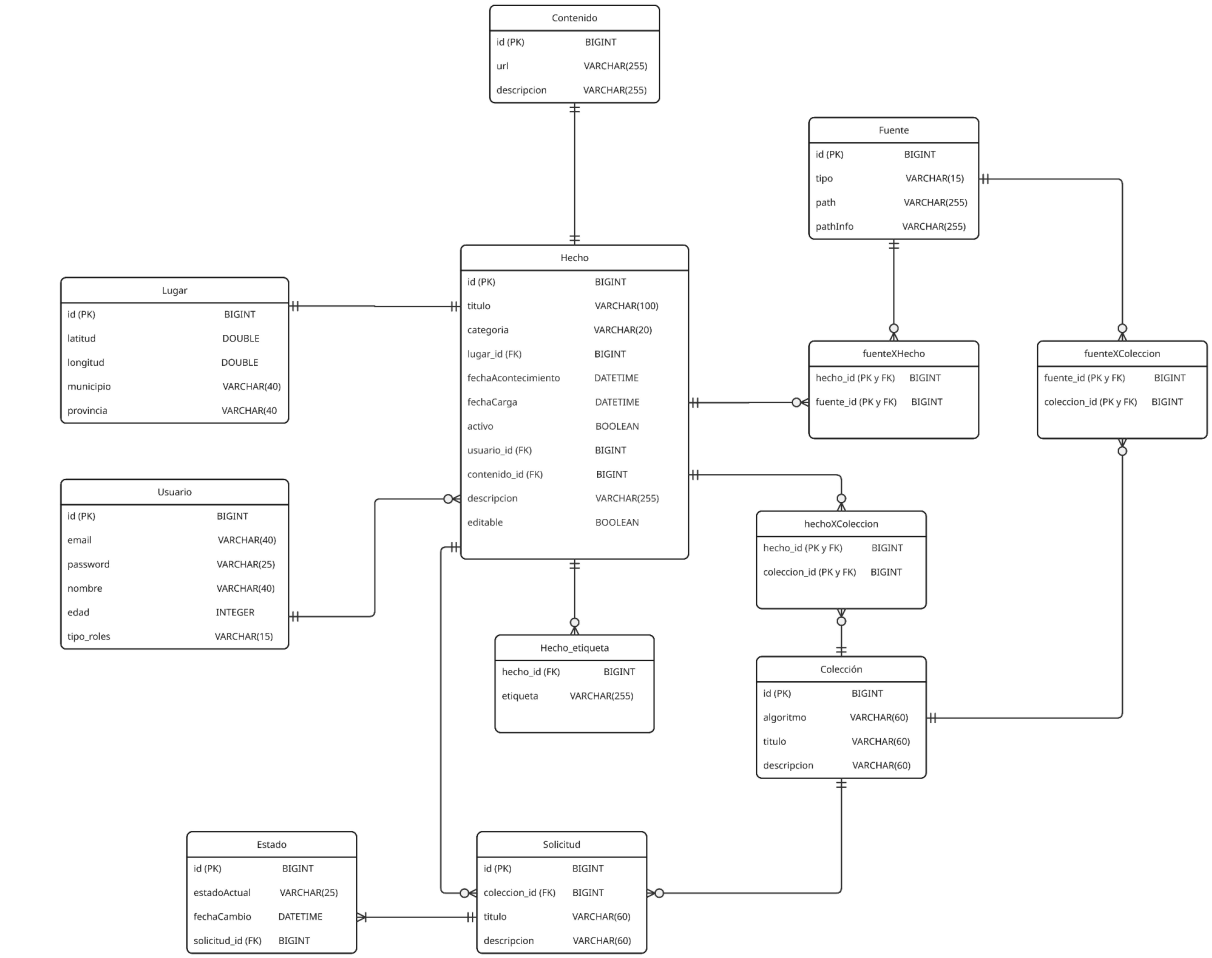
4- Dar todas las estadísticas de una colección:

Devuelve todas las estadísticas asociadas a una colección específica, sin filtrar por tipo
Endpoint: GET /estadisticas/coleccion/{coleccionId}

Respuesta (200 OK) ejemplo:

```
[ {
    "id": 1,
    "tipo": "PROVINCIA_TOP_COLECCION",
    "clave": "Buenos Aires",
    "valor": 120,
    "fechaCalculo": "2025-09-07T00:00:00"
  },
  {
    "id": 2,
    "tipo": "CATEGORIA_TOP_COLECCION",
    "clave": "Deportes",
    "valor": 45,
    "fechaCalculo": "2025-09-07T00:00:00"
  },
  {
    "id": 3,
    "tipo": "SOLICITUDES_SPAM_COLECCION",
    "clave": "spam",
    "valor": 3,
    "fechaCalculo": "2025-09-07T00:00:00"
  }
]
```

Persistencia - DERs



En el DER se pueden observar doce entidades. Consideramos como principal al objeto “Hecho”. Esta, contiene los campos de

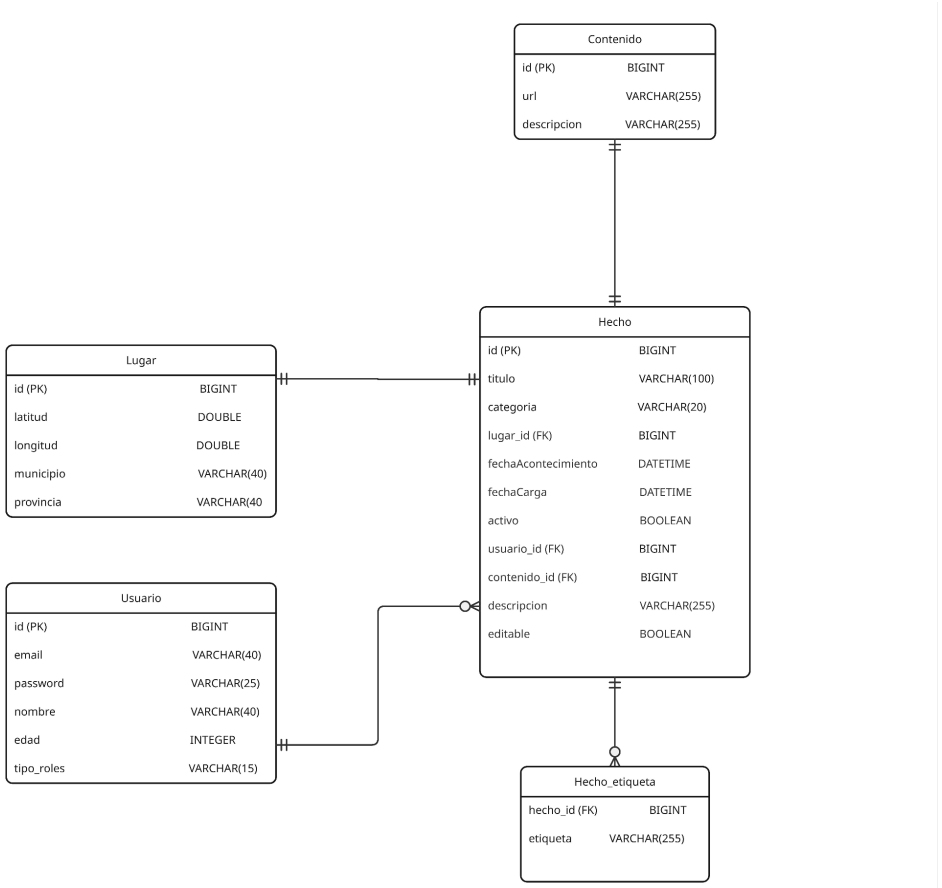
- ID: identificador
- Titulo: Varchar
- Categoria_id: Int [1]
- Lugar_id: Int [2]
- FechaAcontecimiento: LocalDate
- FechaCarga: LocalDate
- Activo: Boolean
- Usuario_id : Int [3]
- Contenido_id: Int [4]
- Descripcion: Varchar
- Editable: Boolean

Almacenamos el id de la categoría dentro del hecho por temas de normalización. No creemos que sea adecuado tener la categoría como un varchar dentro de la entidad ya que puede haber errores de ortografía o consistencia. Además, puede pasar que haya redundancia de datos. Un ejemplo de la entidad hecho puede ser:

1	Titulo1	Inundacion	2	03/03/2005	04/08/2004	True	4	2	“edjsk”	True
2	Titulo2	Inundación	3	14/04/2005	04/01/2005	False	4	1	“miedo”	False
3	Titulo3	inundasión	4	01/09/2004	08/09/1999	True	3	9	“oh por dios”	False

En este caso, los datos no están normalizados ya que los hechos 1, 2 y 3 tienen la misma categoría “Inundación” pero al ser un varchar puede haber inconsistencia de datos, haciendo entonces que se consideran 3 categorías complementamente distintas.

Se puede observar la normalización de los campos lugar, usuario y contenido para evitar repetición de campos, o inconsistencia de datos. Además, ayuda al filtrado de hechos según ciertos atributos.



Esquema Fuente Dinámica

HECHO

Field Types

#	Field	Schema	Table	Type
1	id	fuente_dinamica	hecho	BIGINT
2	activo	fuente_dinamica	hecho	BIT
3	categoria	fuente_dinamica	hecho	VARCHAR
4	descripcion	fuente_dinamica	hecho	VARCHAR
5	editable	fuente_dinamica	hecho	BIT
6	fecha_acontecimiento	fuente_dinamica	hecho	DATE
7	fecha_carga	fuente_dinamica	hecho	DATE
8	titulo	fuente_dinamica	hecho	VARCHAR
9	contenido_id	fuente_dinamica	hecho	BIGINT
10	usuario_id	fuente_dinamica	hecho	BIGINT
11	lugar_id	fuente_dinamica	hecho	BIGINT

ETIQUETA

Field Types

#	Field	Schema	Table	Type
1	hecho_id	fuente_dinamica	hecho_etiqueta	BIGINT
2	etiquetas	fuente_dinamica	hecho_etiqueta	VARCHAR

LUGAR

Field Types

#	Field	Schema	Table	Type
1	id	fuente_dinamica	lugar	BIGINT
2	latitud	fuente_dinamica	lugar	DOUBLE
3	longitud	fuente_dinamica	lugar	DOUBLE
4	municipio	fuente_dinamica	lugar	VARCHAR
5	provincia	fuente_dinamica	lugar	VARCHAR

CONTENIDO

Field Types

#	Field	Schema	Table	Type
1	id	fuente_dinamica	contenido	BIGINT
2	descripcion	fuente_dinamica	contenido	VARCHAR
3	url	fuente_dinamica	contenido	VARCHAR

USUARIO

Field Types

#	Field	Schema	Table	Type
1	id	fuente_dinamica	usuario	BIGINT
2	edad	fuente_dinamica	usuario	INT
3	email	fuente_dinamica	usuario	VARCHAR
4	nombre	fuente_dinamica	usuario	VARCHAR
5	password	fuente_dinamica	usuario	VARCHAR
6	tipo_roles	fuente_dinamica	usuario	ENUM

Esquema Servicio Agregador

COLECCIÓN

Field Types

#	Field	Schema	Table	Type
1	id	servicio_agregador	coleccion	BIGINT
2	tipo_algoritmo	servicio_agregador	coleccion	ENUM
3	descripcion	servicio_agregador	coleccion	VARCHAR
4	titulo	servicio_agregador	coleccion	VARCHAR

FUENTE

Field Types

#	Field	Schema	Table	Type
1	id	servicio_agregador	fuente	BIGINT
2	path	servicio_agregador	fuente	VARCHAR
3	path_info	servicio_agregador	fuente	VARCHAR
4	tipo_fuente	servicio_agregador	fuente	ENUM

Se realizó una tabla pivot con colección x fuente.